

Trees

1. Write an algorithm for counting the number of left external nodes in a binary tree, using the Binary Tree ADT.

left external nodes $\rightarrow$ a node (left child) (leaf node (0 child))

$\therefore$ recursion works well

Plan: • traverse whole tree

- at each node, if left child exists & is external, increment
- otherwise, recurse

base case $\rightarrow$ node doesn't exist

```
int countLL(TreeNode* root) {
    if (root == nullptr) return 0;
    int count = 0;
    if (root->left != nullptr) {
        if (isExternal(root->left)) {
            count += 1;
        } else {
            count += countLL(root->left);
        }
    }
    count += countLL(root->right);
    return count;
}
```

$O(N)$ time completely $\rightarrow$ look at every node

$O(H)$ (call stack)

Height, can be N

2. How do you delete a node with two children in a binary search tree?

Choose an inheritor. Can be:

in-order predecessor (largest in left subtree)

in-order successor (smallest in right subtree)

3. perfect binary tree: both full & complete

full / proper binary tree: each node has 0 or 2 children

complete binary tree: all levels are full except the bottom

balanced binary tree: the height difference b/w left & right is max 1.

4. Rule for Binary search Tree:

5. Rule for AVL Tree :

6. Draw a binary tree T such that

a) each internal node stores one character

b) preorder traversal yields EXAMFUN root $\rightarrow$ left $\rightarrow$ right

c) inorder traversal yields **MAFXUEN** left $\rightarrow$ root $\rightarrow$ right

root is E



7. Design and write an algorithm to traverse and output the post order traversal of a binary tree.

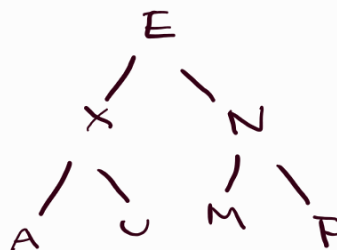
```
void postOrder(TreeNode* root) {
    if (root == nullptr) return;
    postOrder(root->left);
    postOrder(root->right);
    cout << root->val << " ";
}
```

8. Create a binary search tree with the input QWERTYUIOP

9. Insert QWERTYUIOP into an initially empty AVL Tree

10. Represent the following array as a drawn tree, then convert it to a binary search tree, assuming you add elements to an initially empty BST by level order traversal of the given tree. What if you did a pre order traversal?

0	1	2	3	4	5	6
E	X	N	A	U	M	F
✓	└──┘		└──────────┘			



level order = [E, X, N, A, U, M, F]

pre order = [E, X, A, U, N, M, F]

level order BST

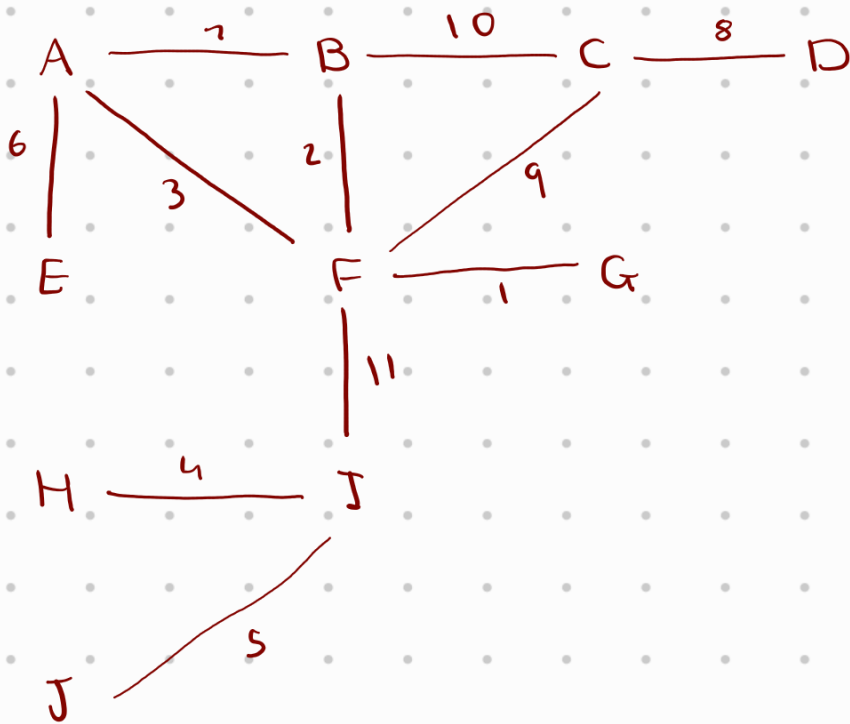


Pre order BST



Graphs

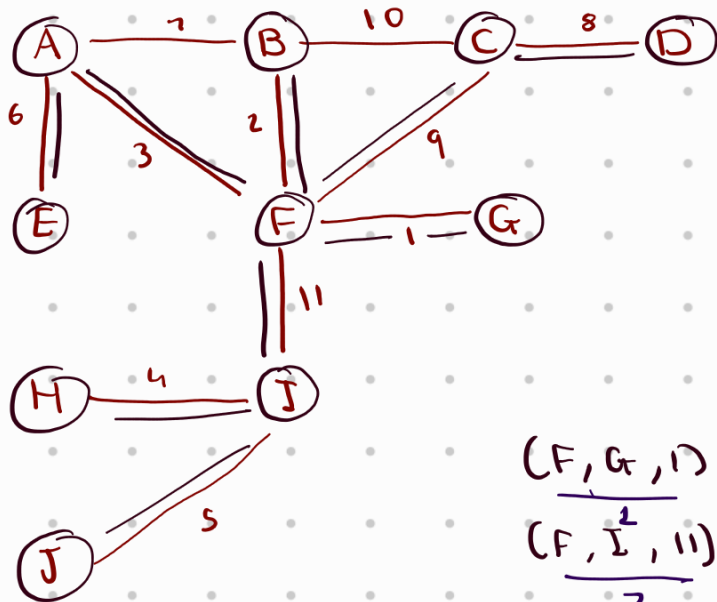
1. Graph Traversal (take smallest edge when applicable)



a) Starting at node A, output the breadth first traversal:

b) Starting at node B, output the depth first traversal:

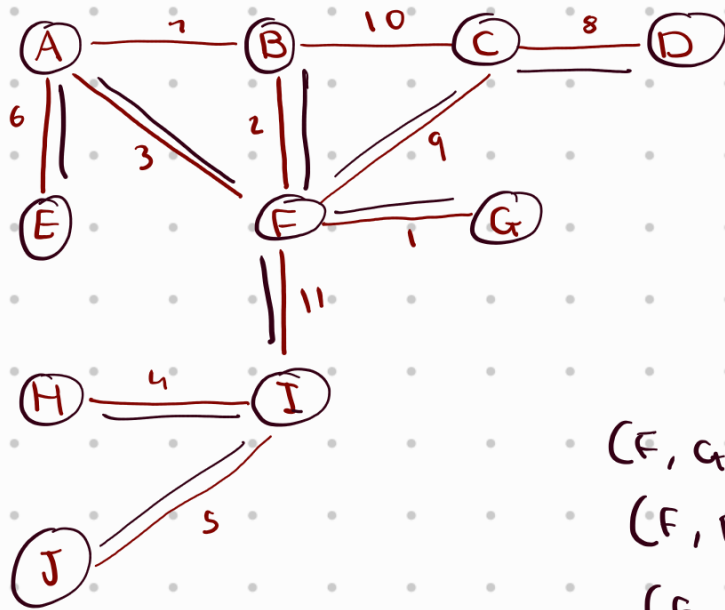
2. Graph Algorithms



(a) Prim's Algorithm
 starting at F, what
 is the MST formed by
 Prim's? (show edges in
 the order they're
 added to the MST)

$(F, G, 1)$ $(F, B, 2)$ $(F, A, 3)$ $(F, C, 9)$
 $(F, I, 11)$ $(B, A, 7)$ $(B, C, 10)$ $(A, E, 6)$
 $(C, D, 8)$ $(I, H, 4)$ $(I, J, 5)$

(F, G) (F, B) (F, A) (A, E) (F, C) (C, D) (F, I) (I, H) (I, J)
 $1 + 2 + 3 + 6 + 9 + 8 + 11 + 4 + 5$



(b) Kruskal's Algorithm

What is the MST formed by Kruskal's algorithm? Show edges in the order they're added.

- | | | |
|-----------|---|------------|
| (F, G, 1) | } | (A, E, 6) |
| (F, B, 2) | | (C, D, 8) |
| (F, A, 3) | | (F, C, 9) |
| (I, H, 4) | | (F, I, 11) |
| (I, J, 5) | | |

3. Graph Terms

a) Incident: edge to vertex if vertex is endpoint

b) Adjacent: vertices A to B if they're connected by one edge

c) Degree: number of edges to or from a vertex

i) in-degree:

ii) out-degree:

d) self loop:

e) Parallel / multiple edges:

f) spanning tree vs. spanning subgraph:

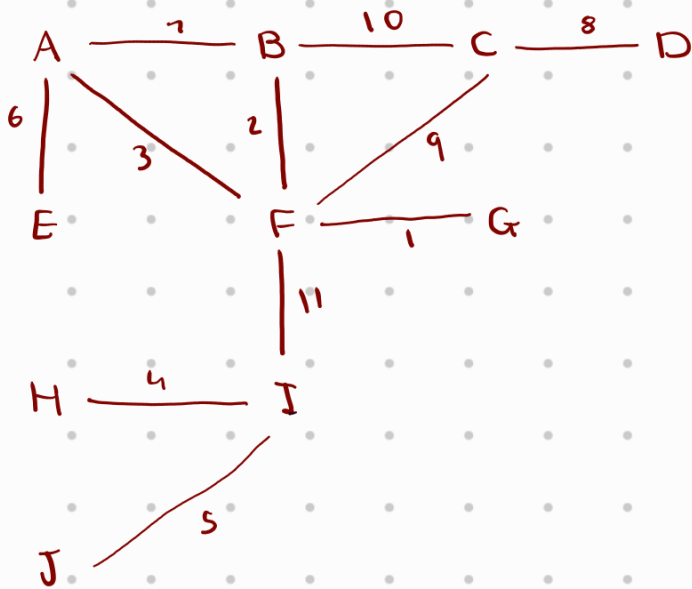
↑

- contains all vertices
- connected
- no self loops

↑

- contains all vertices
- connected

4. Graph Representation



a) Represent the above graph as an adjacency list. Entries can be (adjacent vertex, edge weight).

A: B, E, F

B: A, C, F

C: B, F, D

D: C

E: A

F: A, B, C, G, I

G: F

H: I

I: H, J, F

J: I

Hashes

1. Add the values given, in order, to a hash table of capacity 11 using open addressing, using the hash function and collision handling method providing.

25 14 3 36 12 47

a) $h(x) = x \% 11$, linear probing

0	
1	12
2	
3	25
4	14
5	3
6	36
7	47
8	
9	
10	

$$h(25) = 25 \% 11 = 3$$

$$h(14) = 14 \% 11 = 3$$

$$\hookrightarrow h(14) = (3 + 1) \% 11 = 4$$

$$h(3) = 3 \% 11 = 3$$

$$\hookrightarrow h(3) = (3 + 1) \% 11 = 4$$

$$\hookrightarrow h(3) = (3 + 2) \% 11 = 5$$

$$h(36) = 36 \% 11 = 3$$

$$\hookrightarrow h(36) = (3 + 1) \% 11 = 4$$

$$\hookrightarrow h(36) = (3 + 2) \% 11 = 5$$

$$\hookrightarrow h(36) = (3 + 3) \% 11 = 6$$

$$h(12) = 12 \% 11 = 1$$

$$h(47) = 47 \% 11 = 3$$

$$\hookrightarrow h(47) = (3 + 1) \% 11 = 4$$

$$\hookrightarrow$$

$$\hookrightarrow$$

$$\hookrightarrow h(47) = (3 + 4) \% 11 = 7$$

b) $h(x) = x \% 11$, quadratic probing

0	
1	36
2	12
3	25
4	14
5	
6	
7	3
8	47
9	
10	

25 14 3 36 12 47

$$h(25) = 25 \% 11 = 3$$

$$h(14) = 14 \% 11 = 3$$

$$\hookrightarrow h(14) = (4) \% 11 = 4$$

$$h(3) = 3 \% 11 = 3$$

$$\hookrightarrow h(3) = (4) \% 11 = 4$$

$$\hookrightarrow h(3) = (3+4) \% 11 = 7$$

$$h(36) = 36 \% 11 = 3$$

$$\hookrightarrow 4$$

$$\hookrightarrow 7$$

$$\hookrightarrow 3+9 = 12 \% 11 = 1$$

$$h(12) = 12 \% 11 = 1$$

$$\hookrightarrow h(12) = 2 \% 11 = 2$$

$$h(47) = 47 \% 11 = 3$$

$$\hookrightarrow 3+1 = 4$$

$$\hookrightarrow 3+4 = 7$$

$$\hookrightarrow 3+9 = 12 \% 11 = 1$$

$$\hookrightarrow 3+16 = 19 \% 11 = 8$$

c) $h(x) = x \% 11$, double hashing

$$h_2(x) = 7 - (x \% 7)$$

0	
1	12
2	
3	25
4	
5	47
6	
7	3
8	
9	36
10	14

25 14 3 36 12 47

$$h(25) = 25 \% 11 = 3$$

$$h(14) = 14 \% 11 = 3$$

$$\hookrightarrow h_2(14) = 7 - (14 \% 7) = 7 \mid h(14) = (3+7) \% 11 = 10$$

$$h(3) = 3 \% 11 = 3$$

$$\hookrightarrow h_2(3) = 7 - (3 \% 7) = 4 \mid h(3) = (3+4) \% 11 = 7$$

$$h(36) = 36 \% 11 = 3$$

$$\hookrightarrow h_2(36) = 7 - (1) = 6 \mid h(36) = (3+6) \% 11 = 9$$

$$h(12) = 12 \% 11 = 1$$

$$h(47) = 47 \% 11 = 3$$

$$\hookrightarrow h_2(47) = 7 - 5 = 2 \mid h(47) = (3+2) \% 11 = 5$$

2. a) Primary clustering:

b) Secondary clustering:

c) Load Factor: $\alpha = \frac{n}{m}$

- n - elements stored
- m - capacity (# slots)

3. Insert the following into an empty hash table using separate chaining :

25 14 3 36 12 47 58 8

using the hash function:

$$h(x) = x \% 11$$

Sorting note: stable means it preserves the relative order of equal/identical elements in the array.

1. Sort the following array in alphabetical order using merge sort. What are the time and space complexities? Is it stable? Show your steps.

["sit", "bit", "cat", "net", "dog", "map", "hop"]

2. Sort the following array in alphabetical order using quick sort. What are the time and space complexities? Is it stable? Show your steps, and use the last element as the pivot

["sit", "bit", "cat", "net", "dog", "map", "hop"]